

BASES DE DATOS II

Base de Datos II

Trabajo Práctico — Parcial 2

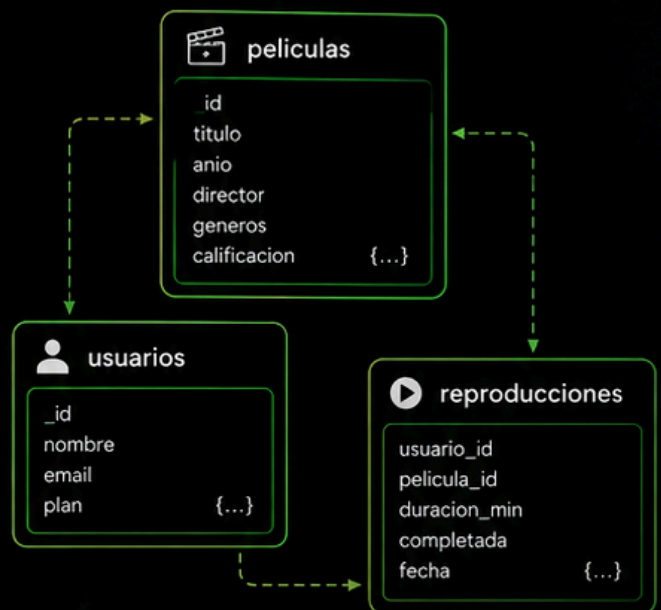


StreamBase — MVP con MongoDB

Tecnicatura Universitaria en Programación | Div133



N



¿Qué hace este programa?

StreamBase es un MVP (Mínimo Producto Viable) que simula el backend de una plataforma de streaming tipo Netflix. Demuestra el modelado de datos en MongoDB usando decisiones fundamentadas de embedding y referencing, y ejecuta tres Aggregation Pipelines reales sobre la base de datos.



GRUPO 14

- Sebastián Lescano
- Joaquín Avallone
- Lautaro Subeldia
- Thiago Martínez
- Mateo Corbetto



MODELADO DE DATOS

Embedding y Referencing



AGGREGATION

3 Pipelines reales de consulta



MONGODB

Colecciones y relaciones



MVP FUNCIONAL

Backend de una plataforma de streaming



PYTHON
3.9 o superior



MONGODB
localhost:27017



PYTHON
seed.py
queries.py



GITHUB
LautaroSantiago/
streambase-bd2



AÑO
2026

Aggregation Pipeline de MongoDB

El Aggregation Pipeline de MongoDB es una forma de trabajar los datos pasando por varios pasos o etapas. La idea es que los documentos entran en una especie de “tubería” (pipeline) y en cada etapa se les aplica una transformación distinta, como filtrarlos, agruparlos, ordenarlos o calcular algún dato.

Cada etapa recibe un flujo de documentos, los procesa en memoria y le pasa el resultado a la etapa siguiente. Es importante destacar que **este proceso es inmutable**: el pipeline no modifica los datos reales en la base de datos, solo transforma la información sobre la marcha para entregar un resultado final.

No es lo mismo que hacer una consulta normal con find(), porque find() sirve más que nada para buscar y traer documentos tal cual están. En cambio, con aggregate() podemos modelar los datos y sacar información mucho más elaborada, como totales, promedios, rankings o estadísticas complejas

Por ejemplo, si tenemos una colección de películas, un documento individual podría verse así:

```
{
  "titulo": "Inception",
  "generos": ["Ciencia Ficción", "Acción", "Suspenso"],
  "director": {
    "nombre": "Christopher Nolan",
    "nacionalidad": "Británica"
  },
  "duracion": 148,
  "reproducciones": 15000,
  "rating": 4.8
}
```

Con una consulta (find), podría traer todas las películas que pertenezcan al género de "Ciencia Ficción":

```
db.peliculas.find({ generos: "Ciencia Ficción" })
```

Pero por ejemplo si quiero saber **cuántas reproducciones en total han acumulado los directores** dentro de ese género específico, una búsqueda simple no alcanza. Ahí es donde entra el Aggregation Pipeline utilizando sus distintas etapas:

\$match (Filtrado): Sirve para filtrar los documentos según un criterio, equivalente al WHERE de SQL. Como **buena práctica de optimización**, \$match siempre debe ir al principio del pipeline. De esta forma se reduce drásticamente el volumen de datos que viaja por la tubería, permitiendo además que MongoDB aproveche los índices de la colección para que la consulta sea muchísimo más rápida.

```
{ $match: { generos: "Ciencia Ficción" } }
```

\$group (Agrupación): Sirve para agrupar los documentos (equivalente al GROUP BY de SQL). Permite acumular y calcular valores. Aquí agrupamos por el nombre del director (usando la ruta del objeto embebido "\$director.nombre") y sumamos todas sus reproducciones.

```
{
  $group: {
    "_id": "$director.nombre",
    "totalReproducciones": { "$sum": "$reproducciones" }
  }
}
```

\$sort (Ordenamiento): Sirve para ordenar los resultados (equivalente al ORDER BY de SQL). Usar -1 significa ordenar de mayor a menor (descendente), ideal para ver primero a los directores más vistos.

```
{ $sort: { totalReproducciones: -1 } }
```

Al unir todas las piezas en el método aggregate(), el código ya queda estructurado en un array con etapas secuenciales.

Primero filtra únicamente las películas de ciencia ficción; luego toma esos documentos, los agrupa por director para calcular cuántas reproducciones acumuló cada uno en ese género y, finalmente, ordena la lista para mostrarnos los directores más exitosos de mayor a menor.

```
db.peliculas.aggregate([
  {
    $match: { generos: "Ciencia Ficción" }
  },
  {
    $group: {
      _id: "$director.nombre",
      totalReproducciones: { $sum: "$reproducciones" }
    }
  },
  {
    $sort: { totalReproducciones: -1 }
  }
])
```

El Aggregation Pipeline es una herramienta indispensable cuando no alcanza con buscar datos, sino que necesitamos analizarlos. Es una solución sumamente potente porque permite estructurar las consultas complejas por etapas legibles, optimizar el uso de memoria y obtener métricas listas para consumir directamente desde el catálogo de películas.